DATASTOCK · REGISTRO DE PRUEBAS

UT2 · Acceso a bases de datos relacionales (JDBC) | Acceso a Datos | 2.º DAM | IES Isidra de Guzmán | Curso 2026–2027

Alumno/a	Edicxon Gabriel López Duque
Repositorio GitHub	https://github.com/edicxonlopezcolab/datastock
Etiqueta de entrega	UT2-v1.0 (último commit de código: 05dad82)
Fecha de ejecución	Del 30 de octubre al 7 de noviembre de 2026
Entorno	Windows (casa) y Linux (instituto) · MySQL 8.0.46 · JDK 21 · MySQL Connector/J 9.3.0 · sqlite-jdbc 3.45.1.0
Resultado global	**37 pruebas ejecutadas · 37 superadas · 0 no superadas · 0 no ejecutadas**

Cada prueba indica los datos de entrada, el resultado esperado y el obtenido. La salida de consola de cada una se guarda en evidencias/jdbcXX/ y evidencias/sqlite/ con el identificador de la prueba como nombre de fichero. Las pruebas EX-JDBC son adicionales a las que exige el enunciado y están numeradas en el orden de los requisitos.

1. Resumen de pruebas

1.1. Por requisito

Requisito	Descripción	Pruebas	Superadas	Evidencias	Commit
RF-JDBC-01	Conectar y comprobar el esquema	3	**3**	evidencias/jdbc01/	ea03937
RF-JDBC-02	Gestionar categorías y proveedores	8	**8**	evidencias/jdbc02/	7c52db7
RF-JDBC-03	CRUD de productos	6	**6**	evidencias/jdbc03/	0ce5074 · 17da4de
RF-JDBC-04	Consultas con ResultSet	4	**4**	evidencias/jdbc04/	3b1fe10 · 05dad82
RF-JDBC-05	Movimientos con procedimiento almacenado	4	**4**	evidencias/jdbc05/	2c46fde
RF-JDBC-06	Consultar historial de movimientos	3	**3**	evidencias/jdbc06/	cc163dc
RF-JDBC-07	Importación controlada desde la UT1 (memoria → MySQL)	3	**3**	evidencias/jdbc07/	4455f93 · 99bde54
RF-JDBC-08	Transacción JDBC de demostración	3	**3**	evidencias/jdbc08/	ced0400
SQLite	Actividad comparativa: gestor embebido (RA2-b)	3	**3**	evidencias/sqlite/	4a323ac
Total		**37**	**37**		

1.2. Detalle de las pruebas

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
RF-JDBC-01 · Conectar y comprobar el esquema				
CP-JDBC-01	Opción 7.1 con db.properties correcto	Conexión y datos de la BD.	[OK] Conexión establecida correctamente. Base de datos datastock, gestor MySQL 8.0.46 y 0 productos almacenados. *evidencias/jdbc01/CP-JDBC-01_conexion_correcta.txt*	**SUPERADA**
CP-JDBC-01b	Opción 7.1 con el servicio MySQL80 detenido (net stop MySQL80)	Mensaje controlado; la aplicación continúa.	[ERROR] no se puede conectar con el servidor MySQL; comprueba que está arrancado. Vuelve al submenú; al arrancar el servicio, conecta de nuevo. *evidencias/jdbc01/CP-JDBC-01b_servidor_detenido.txt*	**SUPERADA**
CP-JDBC-02	Opción 7.1 con una contraseña incorrecta en db.properties	Mensaje controlado sin mostrar la contraseña; la aplicación continúa.	[ERROR] usuario o contraseña incorrectos. Vuelve al submenú sin mostrar la contraseña. *evidencias/jdbc01/CP-JDBC-02_contrasena_incorrecta.txt*	**SUPERADA**
RF-JDBC-02 · Gestionar categorías y proveedores				
CP-JDBC-03	Alta de las categorías 1 Periféricos, 2 Monitores y 3 Audio y de los proveedores 1 Distribuciones Norte (con email) y 2 Suministros Centro (sin email); listados	Altas con [OK] y listados ordenados por id; el email vacío se muestra como «—».	Categorías: tres [OK] y listado 1, 2, 3 (Total: 3). Proveedores: listado inicial «No hay proveedores registrados», dos [OK] y listado con «—» en el proveedor sin email (Total: 2). *evidencias/jdbc02/CP-JDBC-03_alta_y_listado_categorias.txt* *evidencias/jdbc02/CP-JDBC-03_alta_y_listado_proveedores.txt*	**SUPERADA**
CP-JDBC-04	Categoría con id 1 repetido y nombre «Audio» repetido (id 4); proveedor con id 1 repetido y email repetido (id 3); baja de la categoría 1 y del proveedor 1 con productos asociados	Duplicados y bajas bloqueadas rechazados con mensaje en rojo, sin cerrar la aplicación ni borrar productos.	«ya existe un registro con ese identificador» (ids repetidos), «ya existe un registro con ese valor en un campo que no admite repetidos (nombre o email)» y «no se puede eliminar: tiene registros asociados» en las dos bajas. El listado de categorías sigue con 3 registros. *evidencias/jdbc02/CP-JDBC-04_duplicados_categoria.txt* *evidencias/jdbc02/CP-JDBC-04_duplicados_proveedor.txt* *evidencias/jdbc02/CP-JDBC-04_baja_bloqueada_por_productos.txt*	**SUPERADA**
EX-JDBC-01	Listado de categorías (7.2.2) con la tabla vacía	Aviso informativo, no error.	«No hay categorías registradas.» *evidencias/jdbc02/extra_listado_vacio.txt*	**SUPERADA**
EX-JDBC-02	Alta de categoría con id 0, con nombre vacío y con un nombre de más de 60 caracteres	Errores de validación en rojo; no se inserta nada.	«el id debe ser un entero positivo», «el nombre no puede estar vacío» y «el nombre no puede superar los 60 caracteres». *evidencias/jdbc02/extra_validaciones_alta_categoria.txt*	**SUPERADA**
EX-JDBC-03	Consultar categoría por id: 2, 99 y -1 (7.2.5)	Ficha de la categoría, aviso de inexistente y error de id.	«2 | Monitores»; «No existe ninguna categoría con id 99»; [ERROR] el id debe ser un entero positivo. *evidencias/jdbc02/extra_consultar_categoria.txt*	**SUPERADA**

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
EX-JDBC-04	Modificar el nombre de una categoría: con espacios sobrantes, con un nombre repetido (Audio), id 99, cancelando con N y con nombre vacío	Se eliminan los espacios sobrantes; duplicados, inexistente y vacío se rechazan; cancelar no cambia nada.	[OK] con el nombre ya recortado (2 | Pantallas); error de nombre repetido; «no existe ninguna categoría con id 99»; «Modificación cancelada.»; «el nombre no puede estar vacío». El nombre original se restaura al final. *evidencias/jdbc02/extra_modificar_categoria.txt*	**SUPERADA**
EX-JDBC-05	Eliminar la categoría de prueba 9: cancelando con N, confirmando con S, y eliminar el id 99	Cancelar no borra; confirmar borra; un id inexistente informa del motivo.	«Eliminación cancelada.» (la categoría sigue en el listado); [OK] Categoría eliminada; [ERROR] no existe ninguna categoría con id 99. *evidencias/jdbc02/extra_eliminar_categoria.txt*	**SUPERADA**
EX-JDBC-06	Proveedores: email «correo-malo»; modificar cambiando solo el email (Intro conserva el nombre) y solo el nombre (Intro conserva el email); eliminar el proveedor 2, sin productos	Email inválido rechazado; modificación parcial correcta; baja permitida.	[ERROR] el email no tiene un formato válido; dos [OK] Proveedor actualizado; [OK] Proveedor eliminado. El listado final solo contiene el proveedor 1. *evidencias/jdbc02/extra_proveedores_validacion_modificar_eliminar.txt*	**SUPERADA**
RF-JDBC-03 · CRUD de productos				
CP-JDBC-05	Alta de «tec-001», Teclado, 49.90, stock 10, categoría 1, proveedor 1; consulta por código	Producto insertado con el código normalizado y recuperado con todos sus datos.	[OK] Producto creado. Consulta: TEC-001, Teclado, 49.90 €, stock 10, categoría «1 - Periféricos», proveedor «1 - PcPartes». *evidencias/jdbc03/CP-JDBC-05_alta_y_consulta_producto.txt*	**SUPERADA**
CP-JDBC-06	Modificar TEC-001: nombre «Teclado mecánico» y precio 44.90, conservando categoría y proveedor	Cambios guardados y comprobados en MySQL; el stock no se modifica.	[OK] Producto actualizado. En MySQL: Teclado mecánico, 44.90, stock 10. *evidencias/jdbc03/CP-JDBC-06_modificar_producto.txt*	**SUPERADA**
CP-JDBC-07	Alta con el código TEC-001, que ya existe; alta de RAT-002 con la categoría 99	Rechazo por duplicado y por integridad referencial.	[ERROR] ya existe un producto con el código TEC-001; [ERROR] no existe la categoría con id 99. *evidencias/jdbc03/CP-JDBC-07_codigo_duplicado_y_categoria_inexistente.txt*	**SUPERADA**
CP-JDBC-08	Alta y baja de BOR-001 (sin movimientos); baja de TEC-001, que tiene 2 movimientos	Baja permitida en el primer caso y bloqueada por la clave foránea en el segundo.	BOR-001: [OK] Producto eliminado. TEC-001: [ERROR] no se puede eliminar: tiene registros asociados. En MySQL TEC-001 sigue con stock 11 y movimiento_stock conserva sus 2 filas. *evidencias/jdbc03/CP-JDBC-08_baja_permitida.txt* *evidencias/jdbc03/CP-JDBC-08_baja_bloqueada_por_movimientos.txt*	**SUPERADA**
EX-JDBC-07	Alta con el proveedor 99; alta con datos inválidos (código «MAL CODIGO», nombre «AB», precio -5, stock -1, categoría 0, proveedor «x»); consulta de XXX-999	Rechazo con todos los motivos; la consulta de un código inexistente informa sin excepción.	«no existe el proveedor con id 99»; seis errores en rojo (código, nombre, precio, stock, categoría y proveedor); «No existe ningún producto con el código XXX-999». *evidencias/jdbc03/extra_validaciones_alta_producto.txt*	**SUPERADA**

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
EX-JDBC-08	Modificar TEC-001 con precio 12.345 y con categoría 99, y cancelando con N; eliminar XXX-999	Validación al modificar; cancelar no cambia nada; baja de un código inexistente informa.	«el precio admite como máximo dos decimales»; «no existe la categoría con id 99»; «Modificación cancelada.»; «no existe ningún producto con el código XXX-999». *evidencias/jdbc03/extra_modificar_eliminar_producto.txt*	**SUPERADA**
RF-JDBC-04 · Consultas con ResultSet				
CP-JDBC-09	Opción 7.5.1 con 4 productos en MySQL (AUR-004, MON-003, RAT-002, TEC-001)	Listado con JOIN ordenado por código, con los nombres de categoría y proveedor.	Cuatro productos en orden AUR-004, MON-003, RAT-002, TEC-001, cada uno con su categoría y su proveedor (por ejemplo AUR-004 | Audio | PcPartes). Total: 4 productos. *evidencias/jdbc04/CP-JDBC-09_listado_join.txt*	**SUPERADA**
CP-JDBC-10	Opción 7.5.3 con el texto «TECLADO», con «o» y con «xyz»	Coincidencia parcial sin distinguir mayúsculas; sin coincidencias se informa.	«TECLADO» devuelve TEC-001 (Total: 1); «o» devuelve los 4 productos; «xyz» muestra «Sin resultados.» *evidencias/jdbc04/CP-JDBC-10_busqueda_parcial.txt*	**SUPERADA**
CP-JDBC-11	Opción 7.5.4 con umbral 10 (stocks: AUR-004 7, MON-003 3, RAT-002 25, TEC-001 10)	Solo productos con stock < 10, ordenados por stock; TEC-001 (stock 10) no aparece.	MON-003 (stock 3) y AUR-004 (stock 7). Total: 2 productos. *evidencias/jdbc04/CP-JDBC-11_umbral_stock.txt*	**SUPERADA**
EX-JDBC-09	Opción 7.5.4 (stock bajo) con el umbral 0, con 10 y con -5	El 0 se admite (ningún producto tiene stock menor que 0); con 10 salen los de stock menor; un negativo se rechaza.	Umbral 0: «No hay productos que coincidan.» Umbral 10: MON-003 y AUR-004 (Total: 2 productos). Umbral -5: [ERROR] el umbral debe ser un entero mayor o igual que cero. *evidencias/jdbc04/extra_consultas.txt*	**SUPERADA**
RF-JDBC-05 · Movimientos con procedimiento almacenado				
CP-JDBC-12	ENTRADA de 5 unidades de «tec-001» (stock 10) desde 7.6.1	Stock 10 → 15 y movimiento registrado.	[OK] ENTRADA registrada: stock 10 → 15 (cantidad 5). El movimiento 1 queda en movimiento_stock (consulta de CP-JDBC-13). *evidencias/jdbc05/CP-JDBC-12_entrada.txt*	**SUPERADA**
CP-JDBC-13	SALIDA de 4 unidades de «tec-001» (stock 15) desde 7.6.2	Stock 15 → 11 y movimiento registrado.	[OK] SALIDA registrada: stock 15 → 11 (cantidad 4). En MySQL: stock 11 y 2 movimientos (ENTRADA 5: 10→15; SALIDA 4: 15→11). *evidencias/jdbc05/CP-JDBC-13_salida.txt*	**SUPERADA**
CP-JDBC-14	SALIDA de 50 unidades de TEC-001 (stock 11)	Error en rojo; ni el stock ni el historial cambian.	[ERROR] el procedimiento rechazó la operación: Stock insuficiente. En MySQL, stock 11 y COUNT(*) = 2 antes y después. *evidencias/jdbc05/CP-JDBC-14_salida_superior_stock.txt*	**SUPERADA**

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
EX-JDBC-10	Código ZZZ-999; cancelar con N; cantidad «abc»; cantidades 0 y -3; código vacío	El menú valida antes de llamar al procedimiento; no se modifica nada.	Errores en rojo para el código inexistente, el código vacío y las cantidades 0 y -3; «Operación cancelada.»; con «abc» se repite la pregunta. En MySQL, stock 11 y COUNT(*) = 2 tras todas las pruebas. *evidencias/jdbc05/extra_movimientos.txt*	**SUPERADA**
RF-JDBC-06 · Consultar historial de movimientos				
CP-JDBC-15	Opción 7.7.1 con 4 movimientos y opción 7.7.2 con «tec-001»	Orden por fecha e id; el filtro solo muestra TEC-001.	Listado completo con los ids 1 a 4 (Total: 4 movimientos) y las mismas fechas que MySQL. Filtro: ids 1, 2 y 4 (Total: 3). COUNT(*) = 4 tras las consultas. *evidencias/jdbc06/CP-JDBC-15_historial_ordenado_y_filtrado.txt*	**SUPERADA**
CP-JDBC-16	Opción 7.7.2 con AUR-004, que existe y no tiene movimientos	Aviso informativo, no error.	[AVISO] El producto existe pero no tiene movimientos registrados. *evidencias/jdbc06/CP-JDBC-16_producto_sin_movimientos.txt*	**SUPERADA**
EX-JDBC-11	Historial por producto con el código ZZZ-999 y con el código vacío	Errores del menú; el historial es de solo lectura.	[ERROR] no existe ningún producto con el código ZZZ-999; [ERROR] el código no puede estar vacío. COUNT(*) = 4 después de las pruebas. *evidencias/jdbc06/extra_historial.txt*	**SUPERADA**
RF-JDBC-07 · Importación controlada desde la UT1 (memoria → MySQL)				
CP-JDBC-17	datos/pruebas/jdbc07/ ut2_importacion_mixta.json: PRB-001 y PRB-002 válidos; TEC-001 y RAT-002 ya existen; PRB-003 con proveedor 9 y PRB-004 con categoría 9, inexistentes	Se insertan los válidos, se omiten los duplicados y se rechazan los demás con su motivo; la memoria no cambia.	Analizados 6, insertados 2, omitidos por duplicado 2, rechazados 2 («No existe el proveedor con id 9» y «No existe la categoría con id 9»). MySQL pasa de 4 a 6 productos y los 6 siguen en memoria. *evidencias/jdbc07/CP-JDBC-17_importacion_lote_mixta.txt*	**SUPERADA**
CP-JDBC-18	datos/pruebas/jdbc07/ ut2_fallo_forzado.json con FALLO-01, FALLO-02 y FALLO-03; trigger temporal que lanza un error al insertar FALLO-03	Rollback de todo el lote: ninguna inserción queda confirmada.	[ERROR] fallo en la transacción: … Fallo forzado para la prueba CP-JDBC-18. «No se ha confirmado ninguna inserción del lote (rollback aplicado).» Insertados 0. En MySQL, COUNT(*) = 6 y ningún FALLO-%. Después se elimina el trigger temporal (DROP TRIGGER) y SHOW TRIGGERS queda vacío. *evidencias/jdbc07/CP-JDBC-18_fallo_forzado_rollback.txt*	**SUPERADA**
EX-JDBC-12	Opción 7.8 con la memoria vacía; cancelando con N; repitiendo la importación del lote mixto	Aviso; sin cambios; la repetición no duplica ni sobrescribe.	«no hay productos en memoria para importar»; «Importación cancelada.»; repetición: 6 analizados, 0 insertados, 4 omitidos por duplicado y 2 rechazados. *evidencias/jdbc07/extra_importacion_casos.txt*	**SUPERADA**
RF-JDBC-08 · Transacción JDBC de demostración				

ID prueba	Entrada / datos	Esperado	Obtenido	Estado
CP-JDBC-19	Opción 7.9, escenario A: categorías 901 (DEMO-TX-901) y 902 (DEMO-TX-902) en una transacción	Las dos inserciones se confirman con commit y existen en MySQL.	[OK] COMMIT correcto: las categorías 901 y 902 persisten en MySQL. SELECT en MySQL: 901 y 902. *evidencias/jdbc08/CP-JDBC-19_commit_dos_categorias.txt*	**SUPERADA**
CP-JDBC-20	Opción 7.9, escenario B: la categoría 903 insertada dos veces en la misma transacción	La segunda inserción falla por duplicado, el rollback deshace también la primera y 903 no existe.	[OK] ROLLBACK correcto: el duplicado lanzó error 1062 como se esperaba. [OK] El ID 903 no quedó. SELECT en MySQL: 903 no existe. *evidencias/jdbc08/CP-JDBC-20_duplicado_rollback_total.txt*	**SUPERADA**
EX-JDBC-13	Opción 7.9 ejecutada otra vez sin limpiar los ids 901 y 902	La demostración no se ejecuta si los ids de prueba ya existen.	[AVISO] Los IDs [901, 902] ya existen en MySQL: no se ejecuta la demo. Se muestra el DELETE de limpieza limitado a los nombres DEMO-TX-% y no se pide confirmación. *evidencias/jdbc08/extra_id_duplicado_preventivo.txt*	**SUPERADA**
SQLite · Actividad comparativa: gestor embebido (RA2-b)				
CP-SQLITE-01	DemoSQLite sin existir datos/sqlite/productos_demo.db (el Get-Item previo falla)	Se crea el fichero y la tabla, se insertan SQL-001 y SQL-002 y se recuperan dos filas.	[OK] conexión y tabla; Insertados: 2 | Ya existentes: 0; SQL-001 | Producto demo A y SQL-002 | Producto demo B; [OK] el fichero persiste en disco. *evidencias/sqlite/CP-SQLITE-01_primera_ejecucion.txt*	**SUPERADA**
CP-SQLITE-02	Segunda ejecución de DemoSQLite con el fichero ya creado	INSERT OR IGNORE no duplica los registros.	Insertados: 0 | Ya existentes: 2; las mismas dos filas, sin copias. *evidencias/sqlite/CP-SQLITE-02_segunda_ejecucion.txt*	**SUPERADA**
CP-SQLITE-03	Get-Item datos\sqlite\productos_demo.db después de terminar el programa	El fichero persiste con los datos y los recursos JDBC se cierran.	El fichero existe (12288 bytes). Connection, Statement, PreparedStatement y ResultSet se abren con try-with-resources en DemoSQLite y la ejecución termina con BUILD SUCCESS. *evidencias/sqlite/CP-SQLITE-03_cierre_y_fichero_persistente.txt*	**SUPERADA**

2. Fichas detalladas de prueba

Una ficha por requisito, con la prueba más representativa de cada uno (dos en RF-JDBC-07). El resto de pruebas están en la tabla 1.2.

CP-JDBC-02 · RF-JDBC-01 · Conexión con contraseña incorrecta	
Objetivo	Comprobar que, con una contraseña incorrecta en db.properties, la opción 7.1 informa del problema de forma controlada, sin mostrar la contraseña y sin cerrar la aplicación.
Precondiciones	MySQL arrancado. src/main/resources/db.properties con una contraseña incorrecta a propósito (se restaura después de la prueba).
Datos de entrada	Contraseña incorrecta en db.properties.
Pasos ejecutados	1. Ejecutar mvn exec:java. 2. Seleccionar la opción 7 (Gestionar base de datos MySQL). 3. Seleccionar la opción 1 (Comprobar conexión).
Resultado esperado	Mensaje [ERROR] controlado en rojo, sin la contraseña; DataStock vuelve al submenú.
Resultado obtenido	[ERROR] usuario o contraseña incorrectos (error 1045 traducido por ErroresSQL). La aplicación sigue funcionando.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc01/CP-JDBC-02_contrasena_incorrecta.txt · commit ea03937

CP-JDBC-04 · RF-JDBC-02 · Duplicados y bajas bloqueadas por integridad referencial	
Objetivo	Comprobar que se rechazan ids, nombres y emails repetidos y que no se pueden eliminar categorías ni proveedores con productos asociados.
Precondiciones	Categorías 1 Periféricos, 2 Monitores y 3 Audio; proveedor 1 con email ventas@distribuciones.example; productos asociados a la categoría 1 y al proveedor 1.
Datos de entrada	Categoría: id 1 (repetido) y nombre «Audio» con id 4. Proveedor: id 1 (repetido) y email repetido con id 3. Baja de la categoría 1 y del proveedor 1.
Pasos ejecutados	1. 7 → 2 → 1: alta con el id 1 y alta con el id 4 y nombre «Audio». 2. 7 → 3 → 1: alta con el id 1 y alta con el id 3 y el email repetido. 3. Eliminar la categoría 1 y el proveedor 1, confirmando con S. 4. Listar las categorías.
Resultado esperado	Errores en rojo explicados, sin cerrar la aplicación; los registros se conservan.
Resultado obtenido	«ya existe un registro con ese identificador», «ya existe un registro con ese valor en un campo que no admite repetidos (nombre o email)» y «no se puede eliminar: tiene registros asociados» en las dos bajas. El listado de categorías sigue con 3 registros.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc02/CP-JDBC-04_duplicados_categoria.txt, CP-JDBC-04_duplicados_proveedor.txt y CP-JDBC-04_baja_bloqueada_por_productos.txt · commit 7c52db7

CP-JDBC-08 · RF-JDBC-03 · Baja permitida y baja bloqueada por movimientos	
Objetivo	Comprobar que un producto sin movimientos se elimina y que uno con movimientos no, porque la clave foránea protege el historial.
Precondiciones	TEC-001 con stock 11 y 2 movimientos en movimiento_stock; categoría 1 y proveedor 1 existentes.
Datos de entrada	BOR-001 «Producto a borrar», precio 5, stock 1, categoría 1, proveedor 1; y TEC-001.
Pasos ejecutados	1. 7 → 4 → 1: alta de BOR-001. 2. 7 → 4 → 4: eliminar BOR-001 y confirmar con S. 3. 7 → 4 → 4: eliminar TEC-001 y confirmar con S. 4. En MySQL, consultar TEC-001 y el número de filas de movimiento_stock.
Resultado esperado	BOR-001 se elimina; TEC-001 no, y el historial se conserva.
Resultado obtenido	BOR-001: [OK] Producto eliminado. TEC-001: [ERROR] no se puede eliminar: tiene registros asociados. En MySQL, TEC-001 sigue con stock 11 y movimiento_stock conserva sus 2 filas.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc03/CP-JDBC-08_baja_permitida.txt y CP-JDBC-08_baja_bloqueada_por_movimientos.txt · commits 0ce5074 y 17da4de

CP-JDBC-11 · RF-JDBC-04 · Productos con stock inferior a un umbral	
Objetivo	Comprobar que la consulta compara con stock < umbral, sin incluir el valor del umbral, y ordena por stock.
Precondiciones	4 productos en MySQL: AUR-004 (stock 7), MON-003 (3), RAT-002 (25) y TEC-001 (10).
Datos de entrada	Umbral 10.
Pasos ejecutados	1. Seleccionar 7 → 5 → 4. 2. Introducir el umbral 10.
Resultado esperado	Solo los productos con stock menor que 10, ordenados por stock; TEC-001 (stock 10) no aparece.
Resultado obtenido	MON-003 (stock 3) y AUR-004 (stock 7). Total: 2 productos.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc04/CP-JDBC-11_umbral_stock.txt · commit 3b1fe10

CP-JDBC-14 · RF-JDBC-05 · Salida superior al stock disponible	
Objetivo	Comprobar que el procedimiento registrar_movimiento rechaza una salida mayor que el stock y que ni el producto ni el historial cambian.
Precondiciones	TEC-001 con stock 11 y 2 movimientos registrados.
Datos de entrada	SALIDA de 50 unidades de TEC-001, confirmada con S.
Pasos ejecutados	1. En MySQL, anotar el stock de TEC-001 y COUNT(*) de movimiento_stock. 2. 7 → 6 → 2, código TEC-001, cantidad 50, confirmar con S. 3. Repetir las dos consultas en MySQL.
Resultado esperado	Error en rojo; el stock y el historial no cambian.
Resultado obtenido	[ERROR] el procedimiento rechazó la operación: Stock insuficiente. En MySQL, stock 11 y COUNT(*) = 2 antes y después.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc05/CP-JDBC-14_salida_superior_stock.txt · commit 2c46fde

CP-JDBC-15 · RF-JDBC-06 · Historial completo y filtrado por producto	
Objetivo	Comprobar que el historial se muestra ordenado por fecha e id, que el filtro solo muestra el producto pedido y que la consulta no modifica datos.
Precondiciones	4 movimientos: ids 1 y 2 de TEC-001 (entrada de 5 y salida de 4), id 3 de RAT-002 (entrada de 5) e id 4 de TEC-001 (entrada de 1).
Datos de entrada	Filtro «tec-001».
Pasos ejecutados	1. 7 → 7 → 1: consultar todos los movimientos. 2. 7 → 7 → 2 con «tec-001». 3. En MySQL, ejecutar SELECT * FROM movimiento_stock ORDER BY fecha, id y COUNT(*).
Resultado esperado	Orden por fecha e id; el filtro solo muestra TEC-001; el número de movimientos no cambia.
Resultado obtenido	Listado completo con los ids 1 a 4 (Total: 4 movimientos) y las mismas fechas que MySQL. Filtro: ids 1, 2 y 4 (Total: 3). COUNT(*) = 4 después de las consultas.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc06/CP-JDBC-15_historial_ordenado_y_filtrado.txt · commit cc163dc

CP-JDBC-17 · RF-JDBC-07 · Importación de un lote mixto de memoria a MySQL	
Objetivo	Comprobar que la importación clasifica cada producto, inserta solo los válidos y no modifica el inventario en memoria.
Precondiciones	4 productos en MySQL (AUR-004, MON-003, RAT-002, TEC-001); categorías 1 a 3 y proveedor 1 existentes; inventario en memoria vacío.
Datos de entrada	datos/pruebas/jdbc07/ut2_importacion_mixta.json: PRB-001 y PRB-002 válidos; TEC-001 y RAT-002 ya existen en MySQL; PRB-003 con proveedor 9 y PRB-004 con categoría 9, inexistentes.
Pasos ejecutados	1. Menú principal, opción 2: importar el JSON (6 productos en memoria). 2. 7 → 8 y confirmar con S. 3. Opción 4 del menú principal: listar la memoria. 4. En MySQL, COUNT(*) y listado de producto.
Resultado esperado	2 insertados, 2 omitidos por duplicado y 2 rechazados con su motivo; los rechazados no impiden insertar los válidos; la memoria no cambia.
Resultado obtenido	Analizados 6, insertados 2, omitidos 2 y rechazados 2 («No existe el proveedor con id 9» y «No existe la categoría con id 9»). MySQL pasa de 4 a 6 productos (PRB-001 y PRB-002); PRB-003 y PRB-004 no se insertan; los 6 productos siguen en memoria.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc07/CP-JDBC-17_importacion_lote_mixta.txt · commit 4455f93

CP-JDBC-18 · RF-JDBC-07 · Fallo SQL forzado y rollback del lote	
Objetivo	Comprobar que, si falla una inserción del lote, se deshacen todas las del mismo lote y no quedan inserciones parciales.
Precondiciones	6 productos en MySQL. Trigger temporal trg_prueba_fallo (BEFORE INSERT ON producto) que lanza un error SQLSTATE 45000 cuando el código es FALLO-03; no modifica las restricciones del script.
Datos de entrada	datos/pruebas/jdbc07/ut2_fallo_forzado.json con FALLO-01, FALLO-02 y FALLO-03, válidos para Java.
Pasos ejecutados	1. Reiniciar DataStock (memoria vacía) y crear el trigger en MySQL. 2. Opción 2: importar el JSON (3 productos). 3. 7 → 8 y confirmar con S. 4. En MySQL, COUNT(*) de producto y SELECT … LIKE 'FALLO-%'.
Resultado esperado	Los tres productos superan la validación; el fallo de la tercera inserción provoca el rollback de todo el lote.
Resultado obtenido	[ERROR] fallo en la transacción: … Fallo forzado para la prueba CP-JDBC-18. «No se ha confirmado ninguna inserción del lote (rollback aplicado).» Insertados 0 y sin «Importación finalizada». En MySQL, COUNT(*) = 6 y ningún FALLO-%. El trigger se elimina después (SHOW TRIGGERS vacío).
Estado	**SUPERADA**
Evidencia	evidencias/jdbc07/CP-JDBC-18_fallo_forzado_rollback.txt · commit 99bde54

CP-JDBC-20 · RF-JDBC-08 · Rollback de una transacción con inserción duplicada	
Objetivo	Comprobar que un error de clave duplicada dentro de una transacción provoca rollback y deshace también la primera inserción.
Precondiciones	Categorías 1 a 3 en MySQL; ids 901, 902 y 903 libres al empezar (comprobación previa de la demostración). El escenario A se ejecuta antes (CP-JDBC-19).
Datos de entrada	Categoría 903 (DEMO-TX-903) insertada dos veces en la misma transacción.
Pasos ejecutados	1. 7 → 9 y confirmar con S. 2. Escenario B: setAutoCommit(false), insertar 903, insertar 903 otra vez (error 1062) y rollback(). 3. En MySQL, buscar el id 903.
Resultado esperado	La segunda inserción lanza SQLException (1062); el rollback deshace también la primera y 903 no existe.
Resultado obtenido	[OK] ROLLBACK correcto: el duplicado lanzó error 1062 como se esperaba. [OK] El ID 903 no quedó. En MySQL, 903 no existe y 901 y 902 siguen del escenario A.
Estado	**SUPERADA**
Evidencia	evidencias/jdbc08/CP-JDBC-20_duplicado_rollback_total.txt · commit ced0400

CP-SQLITE-02 · SQLite · Segunda ejecución: no se duplican registros	
Objetivo	Comprobar que repetir DemoSQLite no acumula copias gracias a INSERT OR IGNORE y a la clave primaria.
Precondiciones	datos/sqlite/productos_demo.db creado en CP-SQLITE-01 con SQL-001 y SQL-002. Dependencia org.xerial:sqlite-jdbc 3.45.1.0.
Datos de entrada	Los mismos dos registros: SQL-001 «Producto demo A» y SQL-002 «Producto demo B».
Pasos ejecutados	1. Ejecutar mvn exec:java "-Dexec.mainClass=es.iesisidra.datastock.complementarias.DemoSQLite" por segunda vez.
Resultado esperado	Insertados: 0 | Ya existentes: 2 y las mismas dos filas.
Resultado obtenido	Insertados: 0 | Ya existentes: 2. Se recuperan SQL-001 y SQL-002, sin copias; el fichero persiste en disco.
Estado	**SUPERADA**
Evidencia	evidencias/sqlite/CP-SQLITE-02_segunda_ejecucion.txt · commit 4a323ac

3. Incidencias detectadas y correcciones

Problemas encontrados al ejecutar las pruebas o al revisar el código, y cómo se corrigieron. Las pruebas afectadas se repitieron tras la corrección.

ID / prueba	Problema detectado	Corrección aplicada	Resultado tras repetir
RF-JDBC-01 (salida de la aplicación)	Al salir, mvn exec:java esperaba 15 segundos y avisaba de un hilo del conector MySQL sin terminar.	Se añade cleanupDaemonThreads=false a la configuración del plugin exec del pom.xml.	La aplicación termina sin esperas.

ID / prueba	Problema detectado	Corrección aplicada	Resultado tras repetir
CP-JDBC-15 (historial)	Las fechas de movimiento_stock se mostraban con 6 horas de diferencia respecto a MySQL: el PC usa la hora de Bogotá, la URL JDBC fija serverTimezone=Europe/Madrid y getTimestamp(...).toLocalDateTime() convertía entre zonas.	La fecha se lee con getObject("fecha", LocalDateTime.class), sin conversión de zona.	Las horas de la consola coinciden con las de MySQL (por ejemplo 2026-11-04 11:04:18).
CP-JDBC-11 y EX-JDBC-09 (umbral de stock)	La opción 7.5.4 rechazaba el umbral 0 con «el umbral debe ser un entero positivo», pero el enunciado admite un entero mayor o igual que cero.	Se acepta el 0 y solo se rechazan los negativos, con el mensaje «el umbral debe ser un entero mayor o igual que cero».	Umbral 0: «No hay productos que coincidan.»; umbral -5: error; CP-JDBC-11 sigue igual.
RF-JDBC-07 y RF-JDBC-08 (transacciones)	Detectado al diseñar el lote: ProductoDAO.insertar y CategoriaDAO.insertar abrían su propia conexión, de modo que cada inserción se confirmaba por separado y un rollback no habría deshecho nada.	Sobrecargas insertar(Connection, ...) que reutilizan la conexión de la transacción y no la cierran; insertar(...) delega en ellas.	CP-JDBC-18 y CP-JDBC-20: rollback total comprobado en MySQL.
CP-JDBC-19 y CP-JDBC-20 (primera versión de 7.9)	Con los ids 901 y 902 ya existentes, la demostración no avisaba y el escenario A fallaba con error 1062 (el rollback protegió los datos).	Comprobación previa de ids ocupados antes de pedir confirmación; si existen, no se ejecuta y se indica cómo limpiar.	EX-JDBC-13: aviso sin ejecutar ningún escenario.
Evidencias (cabeceras)	Las cabeceras de las evidencias indicaban config/datastock.properties, un fichero que no existe en el proyecto.	Corregido a src/main/resources/db.properties.	Cabeceras coherentes con el código y el README.
CP-SQLITE-01 y CP-SQLITE-02 (observación)	Aparece el aviso SLF4J «Failed to load class StaticLoggerBinder» al usar sqlite-jdbc.	Ninguna: es un aviso informativo; el driver busca un logger y usa uno nulo.	No afecta al resultado: BUILD SUCCESS.

4. Conclusión de la ejecución

Ejecutadas	Superadas	No superadas	No ejecutadas
37	**37**	**0**	**0**

Funcionalidades y observaciones

RF-JDBC-01	Conexión con los datos de la BD (nombre, MySQL 8.0.46 y número de productos). El servidor detenido y la contraseña incorrecta se controlan con un mensaje en rojo, sin mostrar la contraseña ni cerrar la aplicación.
RF-JDBC-02	Altas, listados, modificaciones y bajas de categorías y proveedores con PreparedStatement. Se rechazan ids, nombres y emails repetidos y la baja con productos asociados (error 1451 traducido por ErroresSQL).
RF-JDBC-03	CRUD de productos con BigDecimal y setBigDecimal. El código se normaliza a mayúsculas, los datos se validan con ValidadorProducto, se comprueba que existan la categoría y el proveedor y la baja se bloquea si hay movimientos, sin borrar el historial. El stock solo cambia desde los movimientos.
RF-JDBC-04	Listado con JOIN ordenado por código, búsqueda por nombre con LIKE parametrizado sin distinguir mayúsculas y umbral estricto (stock < umbral, admitiendo 0 y rechazando negativos) ordenado por stock y código. Los resultados se transforman en objetos desde el ResultSet.

RF-JDBC-05	El procedimiento registrar_movimiento se invoca con CallableStatement, sin transacción propia en Java. El stock anterior y el posterior se leen del movimiento registrado, y una salida superior al stock se rechaza sin cambiar el stock ni el historial.
RF-JDBC-06	Historial completo y por producto, ordenado por fecha e id. Se distingue el producto inexistente del que no tiene movimientos y las consultas son de solo lectura.
RF-JDBC-07	La validación se separa de la inserción: los duplicados se omiten, los demás incumplimientos se rechazan con su motivo y solo los aceptados se insertan en una única transacción (setAutoCommit(false), commit y rollback). No se crean categorías ni proveedores ni se modifica el inventario en memoria. El rollback total se demuestra con un fallo forzado.
RF-JDBC-08	Demostración de commit y rollback con los ids 901, 902 y 903 reservados. Comprueba antes que están libres, verifica el resultado desde otra conexión y restaura el autocommit.
SQLite	DemoSQLite es una clase ejecutable independiente: crea el fichero productos_demo.db, inserta con INSERT OR IGNORE sin duplicar y cierra los recursos con try-with-resources. La comparativa con MySQL está en el README.

Todos los requisitos de la UT2 y la actividad comparativa con SQLite están implementados y probados. Funcionalidades pendientes: ninguna.

Fecha: 7 de noviembre de 2026